

STRING / PATTERN MATCHING ALGORITHMS

1) Knuth - Morris - Pratt (KMP) Algorithm

(-) Brute force $O(mn)$ naive algo.

Pattern a b c d a b c
 1 2 3 4 5 6 7

Prefix & Suffix are subsets of the pattern.

(-) Prefix a, ab, abc, abcd

(-) Suffix cdc, bc, abc, clabc

suffix matching prefix

(-) KMP works on the idea if we have any prefix same as suffix in our string / pattern to match.

(-) Is the beginning of a pattern appearing anywhere else in the given string / text.

(-) generate a π / lps (longest prefix which is same as some suffix) table for our pattern to match.

P1: a b c d a b e a b f
0 0 0 0 1 2 0 1 2 0

(a) We start our patterns indexes from 1 onwards.

P2: a b c d e a b f a b c
0 0 0 0 0 1 2 0 1 2 0

P3: a a b c a d a a b e
0 0 1 0 0 1 0 1 2 0 0

P4: a a a a b a a c d
0 1 2 3 0 1 2 0 0

(b) KMP analyses the string & avoids ^{backtracking} ↑

String: a b a b c a b c a b a b a b d
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

pattern	1	2	3	4	5
	a	b	a	b	d
	0	0	1	2	0

idx: 1 2 3

- (i) $\text{cmp } (j+1)$ in pattern with ?intext
- (ii) if $\text{pat}(j+1)$ mismatches with $\text{txt}(i)$ then move both j & i by 1
- (iii) if they don't match then i of the txt stays there we move j backwards to the index in the lps table.
- (iv) Until mismatches keep on occurring & we move to $j=0$ & even there $\text{pat}(j+1)$ mismatches with $\text{txt}(i)$ then only we move i by 1 in the txt string.

$$TC \quad O \quad (n + m)$$

$\uparrow$
search

$\uparrow$
lps table